LABORATORY OBSERVATION/RECORD

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	5
Student Name:	Rowthu Bhaskar Abhiram
Roll No.:	A24126552114
Date:	30-08-2026

1. TITLE

Implement Modules in Node.js

2. AIM

To understand and implement the concept of modules in Node.js, including built-in modules, CommonJS user-defined modules, and ES6 (ECMAScript) modules, and to demonstrate how code can be organized and reused across files using module import/export mechanisms.

3. OBJECTIVE

- To understand what a module is and why modularity is important in Node.js.
- To use a built-in Node.js core module (fs) to perform file operations.
- To create and use a user-defined CommonJS module with require() and module.exports.
- To create and use a user-defined ES6 module with import and export statements.
- To understand the difference between CommonJS and ES6 module systems.
- To execute and verify the output of each type of module.

4. THEORY

In Node.js, a **module** is a reusable block of code whose existence does not accidentally impact other code. Node.js follows a modular architecture, where functionality is split into separate files (modules) that can be loaded and used wherever needed. This makes applications easier to organize, maintain, and reuse.

Node.js supports the following types of modules:

- **Built-in (Core) Modules** — Modules that come pre-packaged with Node.js, such as fs (file system), http, path, and os. They can be used directly by importing them with require(), without installation.
- **User-defined (Local/CommonJS) Modules** — Modules created by the developer in local files. In the CommonJS system, code is exported using module.exports and imported into another file using require().
- **ES6 Modules** — The modern JavaScript module standard. Code is exported using the export keyword and imported using the import keyword. ES6 modules are enabled in Node.js either by using the .mjs extension or by setting "type": "module" in package.json.
- **Third-party Modules** — Modules published by other developers and installed via npm, e.g. express.

The general flow of using a module is: a file defines and exports one or more values (functions, objects, constants) → another file imports those exported values using require() (CommonJS) or import (ES6) → the imported values are used directly in the importing file, as if they were defined locally.

5. ALGORITHM / PROCEDURE

- Create a new Node.js project folder.
- For the built-in module demo: import the core fs module using require('fs').
- Use fs.writeFile() to write text content to a file (example.txt).
- Use fs.readFile() inside the callback to read the file back and print its content.
- For the CommonJS module demo: create a module file (math.js) that defines a function and a constant and exports them using the export keyword (CommonJS-style named exports).
- Create a main file (main.js) that imports the exported items using import and uses them.
- For the ES6 module demo: create a module file (mathUtils.js) that exports functions and a constant using the export keyword.
- Create an entry file (index.js) that imports the exported items and uses them, printing results to the console.
- Run each file using Node.js (node filename.js) and observe the console output.
- Verify the actual output against the expected output for each module type.

6. PROGRAM

(a) Built-in Module Example — builtin.js

Demonstrates use of Node.js's core **fs** module to write and then read a file.

```
const fs = require('fs');

// Write to a file
fs.writeFile('example.txt', 'Hello, Node.js!', (err) => {
  if (err) throw err;
  console.log('File written successfully');

  // Read the file back
  fs.readFile('example.txt', 'utf8', (err, data) => {
    if (err) throw err;
    console.log('File content:', data);
  });
});
```

Output file created — example.txt

```
Hello, Node.js!
```

(b) User-defined Module (CommonJS style) — math.js

Defines and exports a function and a constant.

```
// Exporting a named function
export function add(a, b) {
  return a + b;
}

// Exporting a named constant
export const APP_NAME = 'Simple Calculator';
```

main.js — imports and uses math.js

```
// Import specific items from the module file
import { add, APP_NAME } from './math.js';

console.log(APP_NAME);      // Output: Simple Calculator
console.log(add(10, 20));   // Output: 30
```

(c) ES6 Module Example — mathUtils.js

Defines and exports two functions and a constant using ES6 export syntax.

```
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

// Exporting a constant
export const APP_VERSION = '1.0.0';
```

index.js — imports and uses mathUtils.js

```
import { add, multiply, APP_VERSION } from './mathUtils.js';

console.log(`--- Running App v${APP_VERSION} ---`);
console.log(`Add Result: 10 + 5 = ${add(10, 5)}`);
console.log(`Multiply Result: 4 * 3 = ${multiply(4, 3)}`);
```

7. CODE EXPLANATION

Code	Explanation
`require('fs')`	Imports the built-in Node.js core module used for file system operations.
`fs.writeFile()`	Writes the given content to a file asynchronously; runs a callback on completion.
`fs.readFile()`	Reads the content of a file asynchronously and returns it in the callback.
`export function`	Marks a function as available for import by other files (ES6 module syntax).
`export const`	Marks a constant as available for import by other files (ES6 module syntax).
`import { ... } from './file.js'`	Imports specific named exports from another module file.
`module.exports / export (main.js, math.js)`	Makes the add() function and APP_NAME constant available outside math.js.
`console.log()`	Prints the result of the imported functions/constants to the console.

8. TEST CASES AND EXECUTION DATA

Each module file was executed independently using Node.js, and the console output was recorded.

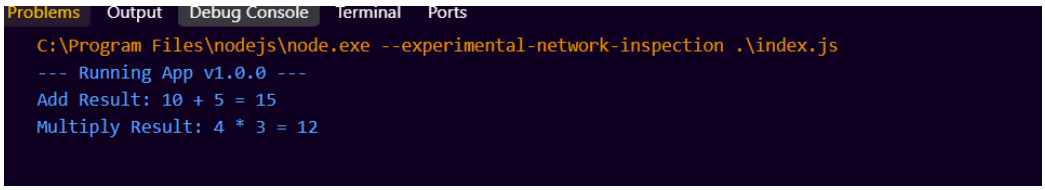
Test Case	File Run	Command	Expected Result	Actual Result	Status
TC-01	builtin.js	node builtin.js	File written, then read back and printed to console	'File written successfully' and 'File content: Hello, Node.js!' printed	Pass
TC-02	main.js	node main.js	Prints APP_NAME and result of add(10, 20)	'Simple Calculator' and '30' printed correctly	Pass
TC-03	index.js	node index.js	Prints APP_VERSION, add(10, 5) and multiply(4, 3)	Version, '15' and '12' printed correctly	Pass
TC-04	example.txt	(generated file)	File contains 'Hello, Node.js!'	File created with exact expected content	Pass

9. OUTPUT

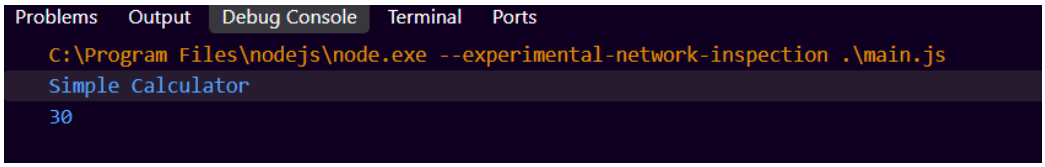
Output 1: Built-in Module (builtin.js)

```
C:\Program Files\nodejs\node.exe --experimental-network-inspection .\builtin.js
File written successfully
File content: Hello, Node.js!
```

Output 2: ES6 Module (index.js)



Output 3: CommonJS Module (main.js)



10. RESULT

Thus, the concept of modules in Node.js was successfully implemented and verified. A built-in core module (fs) was used to write and read a file, a user-defined CommonJS module (math.js) was created and imported into main.js, and an ES6 module (mathUtils.js) was created and imported into index.js. All modules executed correctly and produced the expected output.

How this would be submitted

My final submission would contain:

Source Code

```
Modules-in-NodeJS/
|
|-- builtin.js
|-- math.js
|-- main.js
|-- mathUtils.js
|-- index.js
`-- example.txt
```

PDF

A24126552114_RowthuBhaskarAbhiram_FSWD_Experiment05.pdf
Containing: Title → Aim → Objective → Theory → Algorithm → Program → Code Explanation → Test Cases → Output Screenshots → Result

---

How I would score against the 10-mark rubric

Criterion	Maximum	Example Score
Aim & Objective	1	1
Concept / Theory	2	2
Algorithm / Procedure	1	1
Source Code / Implementation	2	2

Output & Result	2	2
Analysis, Viva & Presentation	2	2
Total	**10**	**10/10**